```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("movie_reviews.csv")
df.head()
```

	review_text	sentiment
0	I loved the movie!	positive
1	It was terrible!	negative
2	Fantastic acting!	positive
3	Worst plot ever.	negative
4	A masterpiece of cinema.	positive

```
df.shape
```

```
(200, 2)
```

```
df["review_text"].value_counts()
```

review_text	count
I loved the movie!	25
It was terrible!	25
Fantastic acting!	25
Worst plot ever.	25
A masterpiece of cinema.	25
Too long and boring.	25
Brilliant visuals!	25
Mediocre at best.	25

dtype: int64

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   review_text  200 non-null    object
 1   sentiment    200 non-null    object
dtypes: object(2)
memory usage: 3.3+ KB
```

```
df.isnull().sum()
```

	0
review_text	0
sentiment	0

dtype: int64

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["sentiment"] = le.fit_transform(df["sentiment"])
df["review_text"] = le.fit_transform(df["review_text"])
```

```
df.head()
```

	review_text	sentiment
0	3	1
1	4	0
2	2	1
3	7	0
4	0	1

```
df.columns
```

```
Index(['review_text', 'sentiment'], dtype='object')
```

```
# Setting the target and Feature Variables
```

```
x = df[['review_text']]
y = df['sentiment']
```

```
from sklearn.model_selection import train_test_split
```

```
xtrain, xtest, ytrain, ytest = train_test_split(x, y, test_size = 0.2, random_state = 42)
xtrain.shape, xtest.shape, ytrain.shape, ytest.shape
```

```
((160, 1), (40, 1), (160,), (40,))
```

```
from sklearn.preprocessing import StandardScaler
```

```
SS = StandardScaler()
xtrain = SS.fit_transform(xtrain)
xtest = SS.fit_transform(xtest)
```

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```

```
model = Sequential()

model.add(Dense(32, activation = "relu", input_shape = (xtrain.shape[1],)))
model.add(Dense(16, activation = "relu"))
model.add(Dense(1, activation = "sigmoid"))
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
# Compiling the model
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
```

```
from tensorflow.keras.callbacks import EarlyStopping
```

```
early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

# Training the model
history = model.fit(
    xtrain, ytrain,
    batch_size=32,
    epochs=500,
    validation_split=0.2,
    callbacks=[early_stopping]
)
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 5.0930e-04 - val_accuracy: 1.0000 - val_loss: 5.8487e-04
```

```
Epoch 425/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 1.0000 - loss: 5.0615e-04 - val_accuracy: 1.0000 - val_loss: 5.8156e-04
Epoch 426/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 1.0000 - loss: 5.0328e-04 - val_accuracy: 1.0000 - val_loss: 5.7831e-04
Epoch 427/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step - accuracy: 1.0000 - loss: 5.0061e-04 - val_accuracy: 1.0000 - val_loss: 5.7502e-04
Epoch 428/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 1.0000 - loss: 4.9767e-04 - val_accuracy: 1.0000 - val_loss: 5.7172e-04
Epoch 429/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 1.0000 - loss: 4.9499e-04 - val_accuracy: 1.0000 - val_loss: 5.6847e-04
Epoch 430/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 1.0000 - loss: 4.9208e-04 - val_accuracy: 1.0000 - val_loss: 5.6523e-04
Epoch 431/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 1.0000 - loss: 4.8931e-04 - val_accuracy: 1.0000 - val_loss: 5.6202e-04
Epoch 432/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 1.0000 - loss: 4.8644e-04 - val_accuracy: 1.0000 - val_loss: 5.5886e-04
Epoch 433/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 4.8390e-04 - val_accuracy: 1.0000 - val_loss: 5.5569e-04
Epoch 434/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 4.8104e-04 - val_accuracy: 1.0000 - val_loss: 5.5253e-04
Epoch 435/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 1.0000 - loss: 4.7826e-04 - val_accuracy: 1.0000 - val_loss: 5.4946e-04
Epoch 436/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 1.0000 - loss: 4.7555e-04 - val_accuracy: 1.0000 - val_loss: 5.4644e-04
Epoch 437/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 1.0000 - loss: 4.7318e-04 - val_accuracy: 1.0000 - val_loss: 5.4334e-04
Epoch 438/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 4.7037e-04 - val_accuracy: 1.0000 - val_loss: 5.4030e-04
Epoch 439/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 1.0000 - loss: 4.6762e-04 - val_accuracy: 1.0000 - val_loss: 5.3731e-04
Epoch 440/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 4.6507e-04 - val_accuracy: 1.0000 - val_loss: 5.3439e-04
Epoch 441/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 1.0000 - loss: 4.6272e-04 - val_accuracy: 1.0000 - val_loss: 5.3145e-04
Epoch 442/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 1.0000 - loss: 4.6016e-04 - val_accuracy: 1.0000 - val_loss: 5.2833e-04
Epoch 443/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 4.5743e-04 - val_accuracy: 1.0000 - val_loss: 5.2547e-04
Epoch 444/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 1.0000 - loss: 4.5489e-04 - val_accuracy: 1.0000 - val_loss: 5.2257e-04
Epoch 445/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 1.0000 - loss: 4.5243e-04 - val_accuracy: 1.0000 - val_loss: 5.1972e-04
Epoch 446/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 1.0000 - loss: 4.4994e-04 - val_accuracy: 1.0000 - val_loss: 5.1697e-04
Epoch 447/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 1.0000 - loss: 4.4748e-04 - val_accuracy: 1.0000 - val_loss: 5.1425e-04
Epoch 448/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 1.0000 - loss: 4.4509e-04 - val_accuracy: 1.0000 - val_loss: 5.1140e-04
Epoch 449/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 1.0000 - loss: 4.4267e-04 - val_accuracy: 1.0000 - val_loss: 5.0851e-04
```

```
# Evaluating the model
loss, accuracy = model.evaluate(xtest, ytest)
print("Accuracy", accuracy)
print("Loss", loss)
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 1s 94ms/step - accuracy: 0.8750 - loss: 0.2934
Accuracy 0.875
Loss 0.29343467950820923
```

```
# Making predictions
y_pred = model.predict(xtest)
y_pred[0:5]
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 128ms/step
array([[1.2656748e-06],
       [1.2656748e-06],
       [9.9999332e-01],
       [9.9999332e-01],
       [9.5640399e-02]], dtype=float32)
```

```
# Converting from Probability
y_pred = (y_pred > 0.5).astype(int)
y_pred[0:5]
```

```
array([[0],
       [0],
       [1],
       [1],
       [0]])
```

```
y_compare = pd.DataFrame(ytest)
y_compare['Predicted'] = y_pred
y_compare = y_compare.reset_index()
y_compare.drop('index', axis = 1)
```

	sentiment	Predicted
0	0	0
1	0	0
2	1	1
3	1	1
4	1	0
5	0	0
6	0	0
7	1	1
8	1	1
9	0	0
10	1	1
11	1	1
12	0	0
13	1	1
14	1	1
15	0	0
16	1	0
17	1	0
18	1	1
19	1	1
20	1	1
21	1	0
22	1	1
23	0	0
24	0	0
25	1	1
26	1	1
27	0	0
28	0	0
29	1	1
30	0	0
31	1	1
32	0	0
33	0	0
34	1	1
35	1	0
36	0	0
37	0	0
38	1	1
39	1	1

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
cr = classification_report(ytest, y_pred)
print(cr)
```

```
              precision    recall  f1-score   support

           0       0.76      1.00      0.86        16
           1       1.00      0.79      0.88        24

    accuracy                           0.88        40
   macro avg       0.88      0.90      0.87        40
weighted avg       0.90      0.88      0.88        40
```